

FORMATION PYTHON

J1

Fondamentaux du langage

Environnement reproductible, types de base et leurs pièges,
conditions, boucles et compréhensions — sur des cas fiscaux réels.

01-premiers-pas

02-controle-de-flux

Objectifs de la journée



Un environnement qui marche

VS Code + uv : Python 3.12 et les bibliothèques, aux versions exactes, pour tous.



Types de base et pièges

int, float, str, bool — dont le piège du float pour les montants en FCFA.



Contrôle de flux complet

if/elif/else, truthiness, for + enumerate/zip, while, break/continue.



Compréhensions

Filtrer et transformer en une expression — liste, dict et set.

LA SEMAINE



Fondamentaux du langage



Fonctions & structures



Fichiers & robustesse



Objets & pandas



Intégration & projets

Fil rouge : la fiche d'un contribuable.

Un dépôt git, un environnement, des données



Le dépôt

Modules numérotés 01→10, TP à trous « À COMPLÉTER », solutions publiées au fil de l'eau, projet final avec socle.



L'environnement

uv sync lit pyproject.toml + uv.lock : même interpréteur, mêmes versions — fini le « ça marche chez moi ».



Les données

Engendrées localement, jamais versionnées : contribuables (NIF), déclarations TVA, paiements — avec défauts volontaires.



démarrage — 3 commandes

```
git clone <adresse-du-depot> formation-python && cd formation-python
uv sync                                # installe Python 3.12 + pandas + jupyterlab
cd 00-data && python generate_data.py --auto # ~50 000 contribuables
```

Le REPL : votre laboratoire permanent



uv run python ouvre le REPL

Chaque idée se teste en 5 secondes avant d'entrer dans le script.



f"{x=}" : nom ET valeur

Le débogage express — affiche ca=45000000 sans rien retaper.



type(), dir(), help()

Quel type ? Que sait-il faire ? Comment s'en servir ? Tout est introspectable.



Erreur = information

Le REPL montre le traceback immédiatement : on apprend en cassant.



le REPL

```
$ uv run python
>>> ca = 45_000_000
>>> print(f"{ca=}")
ca=45000000

>>> type(ca)
<class 'int'>

>>> dir(str)      # toutes les méthodes
['strip', 'title', 'split', ...]
>>> help(str.strip) # doc intégrée
```

Variables : des noms qui pointent vers des valeurs



Le type vient de la valeur

nif = "100004512" est un str : un NIF n'est PAS un nombre (zéros de tête, pas d'arithmétique).



L'affectation ne copie pas

b = a fait pointer deux noms vers le MÊME objet — crucial pour les listes.



Conversion toujours explicite

int("45") + 5 ; str(45) + " FCFA" — Python refuse de deviner, et c'est une qualité.



Nommer, c'est documenter

tva_collectee plutôt que t ou x — le code se lit comme la règle métier.



tp_premiers_pas.py

```
# La fiche d'un contribuable
nif = "100004512"    # str, pas int !
raison = " ets djaló "
ca = 45_000_000      # FCFA
actif = True

nif + 1               # TypeError : tant mieux
int(nif) + 1         # 100004513

# un NIF converti en int perdrait
# ses zéros de tête → toujours str
```

Nombres : int, float — et le piège de l'argent



/ donne TOUJOURS un float

10 / 2 → 5.0. Division entière : // ; reste : %.



Le float est approximatif

0.1 + 0.2 != 0.3 — jamais de == entre floats, jamais de float pour comparer des montants.



Les montants en int (FCFA)

Le FCFA n'a pas de centimes : on calcule, puis round() une seule fois, à la fin.



round() aux demis surprend

round(2.5) → 2 (arrondi bancaire de Python) — à savoir avant un contrôle.



int, float, round

```
0.1 + 0.2
0.30000000000000004    # !

# TVA : calcul float, résultat int
ca = 45_000_000
tva = round(ca * 0.18)
8100000    # int, exact

45_000_000 // 12    # mensualité
3750000
round(2.5), round(3.5)
(2, 4)    # arrondi bancaire
```

Chaînes : nettoyer, découper, tester



La chaîne est immuable

Chaque méthode RENVOIE une nouvelle chaîne : on chaîne les appels — `brut.strip().title()`.



strip / title / upper

Le trio de nettoyage des saisies — " ets djaló " → "Ets Djaló".



split et join, duo inverse

"DEC-2025-0001".split("-") découpe ; "-".join(parts) reconstruit.



Tester avant de convertir

`txt.isdigit()`, `txt.startswith("DEC-")` — la validation commence là.



chaînes

```
brut = " ets fatumata djaló "  
propre = brut.strip().title()  
'Ets Fatumata Djaló'  
  
ident = "DEC-2025-0001234"  
pref, annee, num = ident.split("-")  
annee                # "2025" (str !)  
  
ident.startswith("DEC-") # True  
num.isdigit()           # True  
"12a".isdigit()         # False
```

f-strings : le format sous contrôle



Toute expression entre { }

`f"{ca * 0.18:,.0f}"` — calcul et format directement dans l'affichage.



La spécification après les deux-points

`{ca:,}` milliers · `{taux:.1%}` pourcentage · `{mois:02d}` zéro devant.



Milliers à la française

`{ca:,}` met des virgules → `.replace(",", " ")` pour des espaces.



Alignement pour les listes

`{lib:<18}{montant:>14,}` : des colonnes propres sans bibliothèque.



formats

```
ca, taux = 45_000_000, 0.18
```

```
f"CA : {ca:,} FCFA"
```

```
'CA : 45,000,000 FCFA'
```

```
f"TVA : {taux:.1%}"
```

```
'TVA : 18.0%'
```

```
f"2025-{3:02d}"
```

```
'2025-03'
```

```
for lib, m in lignes:
```

```
    print(f"{lib:<18}{m:>14,}")
```

```
TVA collectée      8,100,000
```


Conditions : le barème, et ce qui est « vrai »



elif : le premier vrai gagne

Ordonner les seuils du plus petit au plus grand évite tout chevauchement.



Truthiness

0, "", [], None sont faux ; le reste est vrai — if telephone: teste « non vide ».



and / or / not

if ca > 0 and regime != "Forfait": — les conditions se composent sans parenthèses inutiles.



Le ternaire pour les cas simples

statut = "actif" if actif else "radié" — une ligne, jamais imbriqué.



tp_bareme.py

```
def regime(ca: int) -> str:
    if ca < 100_000_000:
        return "Forfait"
    elif ca < 500_000_000:
        return "Réal simplifié"
    return "Réal normal"

# truthiness : champ vide ?
if not ligne["telephone"]:
    sans_tel += 1

etat = "OK" if ca > 0 else "KO"
```

for : parcourir SANS gérer d'indice



On itère sur les éléments

for ca in chiffres — jamais for i in range(len(chiffres)).



enumerate quand l'indice sert

enumerate(cas, start=2) : numéro de ligne du CSV (l'en-tête est la ligne 1).



zip pour avancer en parallèle

for nif, ca in zip(nifs, cas) : deux listes, une boucle — s'arrête à la plus courte.



range pour les séquences

range(1, 13) : les 12 périodes mensuelles d'un exercice.

```
enumerate, zip, range

for i, ca in enumerate(cas, start=2):
    if ca <= 0:
        print(f"ligne {i} : CA={ca}")
ligne 5 : CA=0
ligne 9 : CA=-5000000

for nif, ca in zip(nifs, cas):
    print(nif, regime(ca))

for mois in range(1, 13):
    periode = f"2025-{mois:02d}"
```

while, break, continue : piloter la boucle



while : tant que la condition tient

Quand le nombre de tours est inconnu — croissance, réessais, saisie.



break sort immédiatement

Trouvé ? On arrête de chercher — pas un tour de plus.



continue saute au suivant

Ligne invalide ? On la compte et on passe — le corps de boucle reste plat.



Garde-fou anti-boucle infinie

Vérifier que la condition PEUT devenir fausse ; au doute, un compteur plafond.



while & continue

```
# en combien d'années au réel normal ?
ca, annees = 10_000_000, 0
while ca < 500_000_000:
    ca *= 2
    annees += 1
annees == 6

for ligne in lignes:
    if not ligne["nif"]:
        rejets += 1
        continue # au suivant
    traiter(ligne)
```

Compréhensions : liste, dict, set



[expr for x in xs if cond]

Transformer + filtrer en une expression — l’idiome Python n°1.



Dict et set aussi

{c["nif"]: c for c in annuaire} : l’index par NIF en une ligne.



sum() sur une génératrice

sum(1 for ca in cas if ca <= 0) : compter sans liste intermédiaire.



Lisibilité d’abord

Deux for imbriqués ou trois conditions ? Repassez à la boucle classique.



compréhensions

```
# liste : régimes des CA valides
regimes = [regime(ca) for ca in cas
            if ca > 0]

# dict : index nif → fiche
par_nif = {c["nif"]: c
            for c in annuaire}

# set : régions présentes
regions = {c["region"] for c in annuaire}

nb = sum(1 for ca in cas if ca <= 0)
```

Le fil rouge démarre

- 1 **Installer et vérifier** — `INSTALLATION.md`, puis `uv run python main.py`
- 2 **Engendrer les données** — `generate_data.py --auto` — ouvrez les CSV produits
- 3 **`tp_premiers_pas.py`** — fiche formatée : `strip`, `title`, `f-strings` avec `{:,}`
- 4 **`tp_bareme.py`** — `regime()`, comptage des invalides, compréhension finale
- 5 **Pour les rapides** — `while` « années au réel normal » + comptage avec `enumerate`



Fichiers du jour

```
01-premiers-pas/  
    tp_premiers_pas.py  
02-controle-de-flux/  
    tp_bareme.py  
00-data/sortie/*.csv
```



Testez chaque idée au REPL avant de l'écrire dans le fichier. Bloqué 10 min ? Appelez.

À retenir

- ✓ **uv sync** — un environnement identique pour tous — la base du travail en équipe.
- ✓ **NIF = str, montants = int** — les identifiants gardent leurs zéros ; l'argent ne se compare jamais en float.
- ✓ **f"{x=}" et le REPL** — déboguer en montrant nom et valeur ; tester avant d'écrire.
- ✓ **enumerate / zip** — parcourir proprement, sans arithmétique d'indices.
- ✓ **Compréhensions** — transformer + filtrer en une expression — dict et set compris.



Demain — J2 : fonctions et structures de données

Signatures complètes (*args, défauts), portée LEGB, tris avec key=, Counter/defaultdict — sur le calcul des pénalités et le mini-annuaire.